

Technical decisions and the path to production

Shari'ah Compliance Agent, Mal internal compliance tooling.

1. Architecture decisions

The brief asks for an agent, and this is one: it plans its own retrieval, judges whether the evidence it has is enough, and goes back for more when it is not. The decision that shaped everything else is where that agency is bounded.

The model chooses what evidence to gather. It has no say in how the verdict is decided.

It can rewrite a question into the vocabulary the standards use, and it can search again. It cannot touch the control flow, and `DraftAssessment` has no `verdict` field for it to fill in. A compliance verdict has to survive being questioned months later by someone who was not in the room, and an unbounded `ReAct` loop takes a different path on every run.

So the model reports a `finding` about *evidence* (`SUPPORTED_COMPLIANT`, `INSUFFICIENT_BASIS`, `CONFLICTING_SOURCES`) and code maps that onto the `verdict`, which means no code path lets a model produce `COMPLIANT`. Guardrails only escalate toward `NEEDS_REVIEW`, never the other way, so a model regression makes the system noisier rather than more permissive. A wrong `NEEDS_REVIEW` costs a reviewer five minutes; a wrong `COMPLIANT` costs a mispriced product and a regulatory finding. Escalation is therefore the product, not the failure mode: CBUAE reserves Shari'ah determinations to each institution's Internal Shari'ah Supervision Committee, so the metric that matters is not how often the system avoided escalating but how often a non-escalated verdict was wrong.

Qdrant, because it applies payload filters during ANN traversal instead of after it. A query about *ijara* must never be answered from *murabaha* clauses, and across contract families the language is nearly identical: profit distribution under *Mudarabah* and under *Wakala* read alike and mean different things. Ruled out: Chroma, which filters after the search, so recall degrades silently exactly when the filter is doing useful work; pgvector, which leaves hybrid retrieval for me to maintain; Pinecone, proprietary, and with residency rules in play (\$5) self-hostability is not optional.

baai/bge-m3 for embeddings, because Mal's users and Mal's corpus will not always be in the same language, and an English-optimised embedder does not fail loudly on an Arabic query, it returns plausible wrong clauses. Ruled out `text-embedding-3-large`: better in English, worse across languages.

Hybrid retrieval, then a cross-encoder rerank. Dense catches paraphrase; BM25 catches what decides questions in a standards corpus, namely clause numbers like `2/2/2`, transliteration variants, and negation-bearing legal phrases. The arms fuse with Reciprocal Rank Fusion rather than a weighted blend, because cosine and BM25 sit on incomparable scales and any fixed weighting is a constant someone re-tunes whenever the corpus shifts. Reranking then scores query and clause jointly, separating clauses that are topically identical but differ on the one condition that settles the question. It is the dominant cost line, so the backend is pluggable, with `bge-reranker-v2-m3` (Apache-2.0, no egress) the production answer; `jina-reranker-v2-multilingual` scores better and is CC-BY-NC-4.0, so a bank cannot use it.

The clause is the chunk, because the chunk boundary is the citation unit. AAOIFI text is hierarchically numbered (`2/2/2` under `2/2`), so a retrieved chunk is a reference a compliance officer can check by hand, which makes the output auditable rather than merely plausible. Ruled out: fixed windows, which straddle `2/2/2` and `2/2/3` and reduce a citation to a page number; sentence-level, too granular, since "this ratio applies to all *Wakala* deposits" means nothing alone.

No agent framework, just an explicit state machine: `pipeline.py` is 371 lines and the gates in `guardrails.py` another 419. A framework's value is abstraction over many backends, and here the control flow is the thing a reviewer needs to read.

2. Scaling to 50,000 assessments/day

Retrieval infrastructure is not the constraint and will not become one. 50,000/day is 0.6 rps average, 6 rps at a $10\times$ peak, against an index of 1,518 vectors that Qdrant would serve at a thousand times the size.

What breaks first is escalation capacity. Escalation is the product, so every escalated assessment consumes a human. At 10% that is 5,000 reviews/day, roughly 179 full-time reviewers at 15 minutes each; at 25% it is 446. No plausible staffing absorbs that, and Mal's ISSC is a committee rather than a call centre. So the binding constraint is how often the system declines to answer, and it has to reach low single digits for 50,000/day to mean anything. Deduplication therefore stops being an optimisation and becomes load-bearing: if those 50,000 queries hold 3,000 genuinely novel questions, a semantic cache turns an impossible review queue into a feasible one. Escalations also need tiering, since `weak_retrieval` is a corpus-coverage problem for a curator while `insufficient_basis` needs scholarly judgement, and the escalation codes already carry that distinction.

This reorders the model decision. Measured per assessment: 10,317 input and 1,981 output tokens, of which reranking is 54% of the input, which is not where intuition puts it. Haiku 4.5 throughout costs \$1,011/day at target volume, Opus 5 for reasoning \$3,321/day, a gap of \$2,310/day. One extra percentage point of escalation at \$20/review is \$10,000/day. **Break-even is a 0.23 percentage-point change in escalation rate**, so the question is not which model is cheaper per call but which escalates least at a false-COMPLIANT rate of zero. Haiku 4.5 is the default because it is the configuration that was measured, and the comparison against a frontier model is a \$2 experiment the harness is already built for. What it has to watch is calibration: a model that is both weaker and overconfident does not escalate more, it escalates *less*, and the extra answers are wrong.

Model spend is second. A semantic cache keyed to `index_snapshot` saves \$930-1,550/day and invalidates itself for free on re-index, a local reranker saves \$675/day and removes an egress point, and trimming context from 8 clauses to 5 saves \$300/day while usually improving answers. Prompt caching is nearly worthless here, because the retrieved clause text dominating the input varies per query.

Then the request path and state. Each assessment holds a slot for 15-30 seconds against a semaphore of 8 in one process, saturating near 1 rps, so this needs horizontal scale and async by default. The blocker for replicas is the in-process job store, and `jobs.py` is deliberately narrow so Redis is a single-file change. Stale Shari'ah guidance is worse than none because it carries the same confidence, so ingestion moves to a schedule on checksum change, with versioned additive indexing so rollback is a pointer change.

3. Evaluation and quality

Two runs of 40 labelled cases on `claude-haiku-4.5` against index `bff37e1f657b9ae3`, about \$1 each, committed under `eval/results/` with every case's retrieved ids, citations and escalation codes. `scripts/verify_docs.py` re-reads them and fails if these figures drift.

	run 1	run 2
accuracy	0.925	0.925
false COMPLIANT	0	0
over-escalation	3	2
missed escalation	0	1
recall@5 · MRR	0.838 · 0.840	0.811 · 0.870

Zero false COMPLIANT is the result that matters, and it held across both runs and across a third in which a rate limit had broken every upstream call: a failing dependency degrades to escalation by construction. Every failure in both runs was in the guardrails rather than in retrieval or reasoning, meaning the governing clause was retrieved and the model reasoned correctly in each one. The two runs shared a headline and differed underneath, which is why both are reported: at n=40 the stable signal is *which gate fires*, not the aggregate.

One fix was a genuine trade. Escalating on the assessment's own findings caught q027, a capital guarantee phrased as "cover any capital loss" that no query pattern matched, and the same change swallowed q009, where SS12-5.7 prohibits the arrangement flatly and `NON_COMPLIANT` is a good answer. Both turn on a capital guarantee; the difference is that one has a clause settling it outright and the other is a product-design question, and no deterministic signal separates them, because "does a retrieved clause flatly prohibit this" is exactly the judgement being asked of the model. I took

the trade toward over-escalation and left the q009 label alone, because editing a test to match the code is how an eval stops meaning anything.

Retrieval and generation are measured separately, the most important structural choice, because the two failures have different fixes and look identical from the output. Retrieval eval needs no model and runs on every re-index: a gap between recall@10 and recall@3 is ranking, low recall@10 is candidate generation, and context precision measures the noise riding along.

The headline verdict metric is **false-COMPLIANT rate, not accuracy**, because accuracy averages three errors that are not alike. The escalation mix by code is the diagnostic beside it: `low_confidence` dominating means the threshold is miscalibrated, `always_review_category` dominating means the product is pointed at questions it was not built for. Three test sets stay separate: clause-derived cases that sit too close to the source and flatter the retriever, adversarial near-miss pairs differing on one decisive condition, and a production-sampled set that starts empty and becomes the most valuable one.

The escalation queue is the labelling pipeline. Every `NEEDS_REVIEW` already goes to a human who makes a determination, so capturing it costs one field in the review UI and yields perfectly distributed eval data for free. Beside it, a blind audit of a 5% `COMPLIANT` sample, the only way to measure false-COMPLIANT in production. The hard gate on every change is that false-COMPLIANT must not increase; other metrics trade, that one does not.

4. Observability and debugging

Structured JSON to stdout, one event per stage, every line carrying `trace_id` and principal, span names already OpenTelemetry-shaped. Per assessment one `audit.record` event pins corpus version, index snapshot, model and prompt version, and carries every retrieval round with its rationale, every retrieved clause with its per-arm scores, the draft, the final verdict, which guardrails fired, latency and token usage. That record is what makes an assessment replayable months later.

Production adds an OTel collector fanning out to Tempo, Prometheus and Loki; an LLM-specific layer such as Langfuse, because generic APM shows a slow span and not which clause the model ignored; and audit records on append-only WORM storage, separate from operational logs, since they carry a regulatory retention obligation and a different access profile.

The alarm I care most about is **the `NEEDS_REVIEW` rate dropping more than 20% week-over-week**, because it reads like an improvement and is usually a weakened guardrail. Also worth alerting: any escalation code's share shifting sharply, downward drift in top-1 rerank score, a rising `unresolved_citation` rate (model regression, or the gateway quietly rerouting to a different model), and any open circuit breaker.

Debugging a wrong verdict. Pull the audit record by `assessment_id`, which pins the corpus and prompt versions, so a re-index since the assessment is immediately visible. Then split the failure, which is the whole reason retrieval is logged separately and is the first question to ask rather than the last: **was the governing clause in retrieved[]?** If not, replay retrieval against the same snapshot, where per-arm scores separate candidate generation from ranking and `retrieval_rounds[]` shows why the model issued each search, making a bad reformulation visible as a prompt fix. If it was retrieved, the model misread real text rather than fabricating, since quotes were verified verbatim. Either way, ask why no guardrail caught it, because a wrong `COMPLIANT` that passed every gate is a gap in the gates, and add the case to the adversarial set.

5. Security and regulatory compliance

Cross-border transfer of consumer data. CBUAE's Consumer Protection Regulations require licensed institutions to store and process consumer and transaction data inside the UAE, and PDPL (Federal Decree-Law 45/2021) restricts transfer to adequacy-listed jurisdictions. This deployment sends query text to a US-hosted gateway. PII redaction runs on the request path before egress, deliberately not on the logging path, because scrubbing logs while sending raw text upstream protects the wrong artefact. It is a compensating control rather than a sufficient one, being pattern-bound: "the largest depositor at our Sheikh Zayed Road branch" identifies a person while matching nothing. Production moves inference in-country, where CBUAE's sovereign financial cloud with Core42, Azure UAE North and AWS `me-central-1` are options. Embeddings and reranking can go local today, removing two of the three egress points, which is why both sit behind a swappable interface.

Prompt injection through the corpus. AAOIFI text is trusted today. That stops being true the moment Mal indexes its own term sheets or customer submissions, and an instruction buried in a term sheet could aim to flip the verdict on the product that term sheet describes. Three controls are in place: the system prompt states that retrieved text is reference material and never instruction, output is schema-constrained so the model cannot emit free-form actions, and citations must resolve to clauses retrieved this run with quotes appearing verbatim, so a fabricated justification fails the gate without a second model call. The next tier is retrieved content in a structurally distinct channel plus per-chunk provenance with a trust tier. The control that holds regardless is that **a verdict causes no action**: nothing downstream executes on `COMPLIANT`, it routes to a human queue, so an injection that flips a verdict wins a wrong line in a memo a scholar reads.

Authorisation and audit integrity. Authorisation is checked before the pipeline runs rather than inside it, because by the time a tool call exists the decision to permit retrieval and spend must already be made; the model is untrusted input. Jobs are readable only by the principal that created them, so a job id is not a capability, and `assert_production_safe()` inspects the risky settings at startup. The demo uses static bearer tokens; production means OIDC against the bank's IdP with group claims mapped onto the existing `scopes` shape, which changes nothing downstream of `deps.py`, plus per-assessment ACLs tied to the requesting business unit and short-lived credentials with per-principal spend caps.

Automation bias. The realistic failure is not a jailbreak, it is a product manager screenshotting a `COMPLIANT` verdict and treating it as Shari'ah approval. Every response carries a disclaimer reserving determinations to the ISSC, and `always_review_category` catches requests phrased as seeking approval. The rest belongs in the UI as much as the API: `COMPLIANT` feeds an ISSC queue rather than a ship button, and it should never render without its citations beside it.

6. What I deprioritised, and what it would cost

Supersession data. `superseded_by`, the guardrail reading it and the retrieval filter excluding it are all built; populating them needs edition-over-edition diffing the 2017 snapshot does not support. Until then a repealed clause would be cited as though live, which is the first gap I would close.

The second model run. The \$2 comparison decides the bill, and the harness, test set and metrics are all in place to run it for about \$2.70.

Local reranking as the default, implemented and one environment variable away, left off so the project runs on a single API key. Turning it on saves ~\$675/day at target volume.

Durable job state and OIDC, both deliberate demo scaffolding with the seams already cut: `jobs.py` is narrow enough that Redis is a one-file change, and the `scopes` shape downstream of auth is already what an IdP would populate.

Semantic caching, the largest saving left at \$930-1,550/day, held back because deciding what counts as "the same question" is a correctness decision that needed the eval framework to exist first.

Corpus coverage, and a parser bug I would fix before anything else. The index holds 1,518 clauses across 48 of the edition's 54 standards. Standards 42, 43, 44, 46, 47 and 48 are in the source and absent from the index, because the header regex required Standard No. with a period while the running page headers omit it, so their titles never resolved and the quality gate dropped them as damaged. The fix is in `parse.py` with a regression test; the index is not yet rebuilt. This is the clearest example of the failure class the architecture cannot see: a wakala question has no SS 46 to retrieve, so it draws plausible neighbours from SS 40 and SS 23 that score well, `weak_retrieval` stays quiet, and the audit record cannot say that the governing standard was never indexed. It also bounds the eval, whose 40 cases were written from indexed clauses and therefore cannot test a standard that is not there. Beyond this, CBUAE circulars, HSA resolutions and Mal's own product policy are the other half of what governs a real product decision, and none is public.

A public URL. Free container hosting without a payment method has closed: Render, Koyeb and Fly require a card, Hugging Face Spaces made the Docker runtime paid in July 2026, and Back4App's free URLs expire an hour after issue. The deployment path is built and was exercised end to end against a Qdrant Cloud cluster, and `scripts/bootstrap.sh` restores the committed index, so `docker compose up` gives a reviewer the full system with the index loaded in about two seconds, without re-embedding anything.